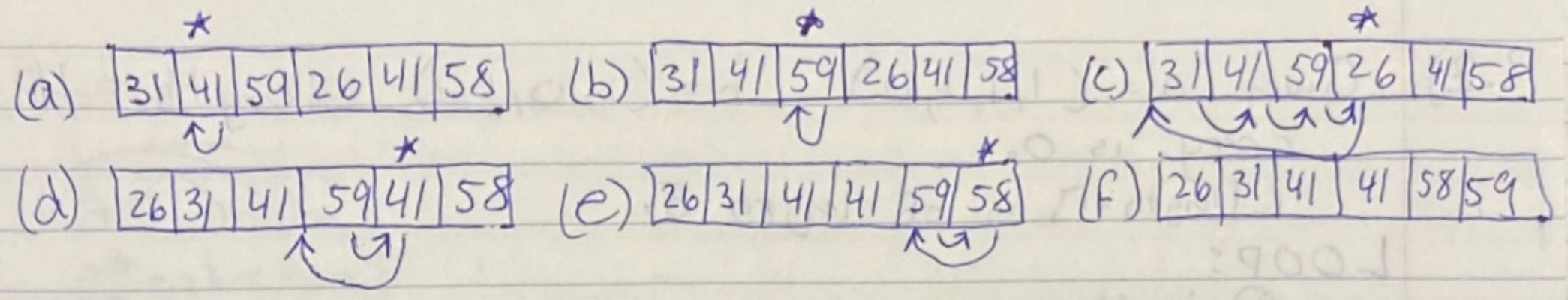
1



* denotes current key being investigated.
↺ denotes where while loop stops.
↳ denotes consequent shifting from loop.

Insertion-Sort // nonincreasing - <u>just change one relational op.</u>

```
for j = 2 to A.length
    key = A[j]
    i = j-1
    while i > 0 and A[i] < key        // just change > to <
        A[i+1] = A[i]
        i--
    A[i+1] = key
```

this is valid because a value is considered to be in the wrong position if it is greater than its predecessors. This will ensure nonincreasing reorder.

2) Add-Binary(a[], b[], length)

```
let c be a new int array of size length+1
let carry = 0

for i = length downto 1
    let sum be [a[i-1] + b[i-1] + carry]
    c[i] = sum % 2
    carry = sum / 2

c[0] = carry // last carry becomes the first bit
return c // return the new array of bits
```

side work

rules

if sum	then
0+0+0	0 c 0
0+0+1	1 c 0
0+1+1	0 c 1
1+1+1	1 c 1

this can be represented as sum%2 c sum/2

~~Ex) given a = $(1110)_2$, b = $(1011)_2$, length = 4~~
~~i = 4: c[4] = (0+1+0) = 1, carry =~~

illustrated on next page

2.2) Given $a = (1,1,1,0)$ $b = (1,0,1,1)$ length $= 4$

carry is 0.

c[length+1] is of length 5.

LOOP:

① $i = 4$.

sum $= (a[3] + b[3] + 0) = (0+1+0) = 1$.

sum %2 $= 1 \% 2 = \underline{1 = c[4]}$.

sum / 2 $= 1/2 = \underline{0 = \text{carry}}$.

$3 > 0$, so loop again.

② $i = 3$.

sum $= (a[2] + b[2] + 0) = (1+1+0) = 2$.

sum %2 $= 2 \% 2 = \underline{0 = c[3]}$.

sum / 2 $= 2/2 = \underline{1 = \text{carry}}$.

$2 > 0$, so loop again.

③ $i = 2$.

sum $= (a[1] + b[1] + 1) = (1+0+1) = 2$.

sum %2 $= 2 \% 2 = \underline{0 = c[2]}$

sum / 2 $= 2/2 = \underline{1 = \text{carry}}$

$1 > 0$, so loop again.

④ $i = 1$.

sum $= (a[0] + b[0] + 1) = (1+1+1) = 3$

sum %2 $= 3 \% 2 = \underline{1 = c[1]}$

sum / 2 $= 3/2 = \underline{1 = \text{carry}}$

$0! > 0$, so <u>do not</u> loop again.

Exit:

$c[0] = 1$, the last carry.

<u>returns:</u>

c[0]	c[1]	c[2]	c[3]	c[4]
1	1	0	0	1

Verify: 11001 represents 25 (16+8+1) which is in fact equal to 1110 + 1011 (14 + 11).

(8+4+2 8+2+1)

3 Express in Θ- notation:

a) $(n^2+8)(n+1)$

$\underline{n^3}+n^2+8n+8$:

$= \Theta(n^3)$ as n^3 is the highest order term.

b) $(n\log n + n^2)(n^3+2)$

$n^4\log n + 2n\log n + \underline{n^5} + 2n^2$

$= \Theta(n^5)$ as n^5 is the highest order term.

c) $(n! + 2^n)(n^3 + \log(n^2+1))$

$\underline{n!n^3} + n!\log(n^2+1) + \underline{2^n n^3}\ 2^n\log(n^2+1)$

$\Theta(n!n^3)$, because it is highest order due to the fact that $n! > 2^n$ for all $n>3$

4 also, both arrays must contain the same elements (can be dif order).

a) In order to prove bubble sort works, it must also be true that A.length is = to A'.length. If the two arrays do not contain equal elements, some elements were lost or extraneous.

b) An invariant for the inner loop is that A[j] is the smallest element.

Initialization: The subarray only contains a single element at n (the last elem of original A), thus (given n=A.length) A[n] is certainly the smallest as it is the only element

Maintenance: Each iteration will compare A[j] to A[j-1]. If the first element is no longer the smallest, they will swap to ensure it still is.

Termination: Since now A[i] is the smallest, and i=j at termination, the first element is thus still the smallest in the array.

c) Outer Loop invariant can be that subarray A[1, i-1] is sorted. (small to big)

Initialization: Initially the subarray is empty, and trivially sorted.

Maintenance: Each time inner loop finishes, A[i] is the ith smallest element from i to n. The next smallest element is added, thus still sorted.

Termination: the largest element is added last, thus the list is still sorted as described by the previous step.

5) Algorithm to locate integers; Let A be integer array list
Let L be array list of locations; n be length of integer array list

```
for i=1 to n
    if (A[i] == x)
        L.add(i)
return L
```

<u>Comparisons:</u> each time if statement condition is checked. this will occur each iteration of the loop; <u>n times.</u>

6) Prove $n^3-91n^2-7n-14$ is $\Omega(n^3)$.

find c, n_0 such that $0 \le cn^3 \le n^3-91n^2-7n-14$ for all $n > n_0$.

NOTE: $cg(n)$ will only ever be less than $f(n)$ if c is fractional, because $f(n)$ is decreasing and $f(n)$ & $g(n)$ are both of order 3. Thus, I chose a fractional c; $\frac{1}{2}$.

<u>note</u> $0 \le cn^3$ always, for $c, n \ge 0$. So I omit this:

$c = \frac{1}{2}$, so:

$$2\left(\frac{1}{2}n^3\right) \le (n^3-91n^2-7n-14)2$$

$$n^3 \le 2n^3-182n^2-14n-28$$

$$-n^3 \qquad -n^3$$

$$0 \le n^3-182n^2-14n-28$$

New Question: when will R.H.S. be non-negative?
this will occur when n^3 exceeds the other terms.
I estimate by the following:

$$0 = \cancel{n^2}\left(n-182-\overbrace{\frac{14}{n}-\frac{28}{n^2}}^{c_2}\right)$$

$$0 = n_0-182-c_2 \rightarrow \underline{n_0 = 182+c_2}$$

I know $\frac{14}{n}-\frac{28}{n^2}$ will be an extremely small constant because n ~~us~~ must exceed 182 for my assertion to hold.
Since $0 < c_2 < 1$, I can assert that the smallest n_0 which satisfies $cg(n) \le f(n)$ for $c = 1/2$ is 183, because 182 is not quite true (but close).

* Thus, $f(n)$ is $\Omega(n^3)$ when $c = 1/2$, $n_0 = 183$.
$(c, n_0) = (1/2, 183)$

$n^2(n-182)$

$n-182=0$

7) Prove $27n^2+18n = \Theta(0.5n^2-100)$

I will first prove $f(n) = \Omega(g(n))$, with c_1 and potential n_0.

$c_1 g(n) \leq f(n)$; $g(n)$ already seems slower, so I try $c_1 = 1$.

$\therefore 0.5n^2-100 \leq 27n^2+18n$

$2(0.5n^2 \leq 27n^2+18n+100)$

$n^2 \leq 54n^2+36n+200$

$-n^2 \qquad -n^2$

$0 \leq 53n^2+36n+200$

For what n_0 does RHS become positive?

$n_0 = 0$, because any $n>0$ is positive and satisfies.

but, proving big-O may narrow this domain.

Big-O: $c_2 g(n) \geq f(n)$; in order for $f(n)$ to become smaller than $c_2 g(n)$, I plan to find c_2 such that I can subtract over and combine n^2 terms, such that the RHS becomes negative, thus less than the ~~right~~ left side, 0, for $n>n_0$.

I choose c_2 to be 56, because I want $27-\frac{1}{2}c_2 < 0$.

27-28 is negative, so lets try $c_2 = 2(28) = 56$.

$56g(n) \geq f(n)$; $56(0.5n^2-100) \geq 27n^2+18n$

$28n^2-5600 \geq 27n^2+18n$

$28n^2 \geq 27n^2+18n+5600$

$-28n^2 \qquad -28n^2$

$0 \geq -n^2+18n+5600$

$n^2 \geq 18n+5600 \quad = 0 \leq n^2-18n+5600$

$a=1 \quad b=-18 \quad c=-5600$

when will n^2 exceed $18n+5600$?

(reject negative result)

$n = \frac{-b \pm \sqrt{b^2-4ac}}{2a} \qquad \frac{18 \pm \sqrt{324+4(5600)}}{2} \rightarrow \frac{18+\sqrt{22724}}{2} \approx \frac{18+\sqrt{22500}}{2}$

here, I recognize $\sqrt{22,724}$ to be close to $\sqrt{22,500}$, a perfect square

using guess/check, 100^2 was too small, 200^2 too big, 150^2 is 22500 - close!

Since I know approximating $\sqrt{22724}$ to be 150 will be just slightly too small to give n_0, I'll add one after.

I get $n \approx \frac{18+150}{2} = 84$, and add one since my est. was low.

* Thus, $27n^2+18n$ is $\Theta(0.5n^2-100)$ for $(c_1, c_2, n_0) = (1, 56, 85)$

8) Strassen (A, B)

$n = A.rows$

let C be new $n \times n$ mtx

if $n == 1$

$C_{11} = a_{11} \cdot b_{11}$

else partition the matrices into 4 quadrants size $\frac{n}{2}$.

let $S_1 \rightarrow S_{10}$ be new $\frac{n}{2} \times \frac{n}{2}$ mtcs

let $P_1 \rightarrow P_7$ be new $\frac{n}{2} \times \frac{n}{2}$ mtcs

$S_1 = B_{12} - B_{22}$

$S_2 = A_{11} + A_{12}$

$S_3 = A_{21} + A_{22}$

$S_4 = B_{21} - B_{11}$

$S_5 = A_{11} + A_{22}$

$S_6 = B_{11} + B_{22}$

$S_7 = A_{12} - A_{22}$

$S_8 = B_{21} + B_{22}$

$S_9 = A_{11} - A_{21}$

$S_{10} = B_{11} + B_{12}$

Assume separate functions for matrix math / combination.

$P_1 = strassen(A_{11}, S_1)$

$P_2 = strassen(S_2, B_{22})$

$P_3 = strassen(S_3, B_{11})$

$P_4 = strassen(A_{22}, S_4)$

$P_5 = strassen(S_5, S_6)$

$P_6 = strassen(S_7, S_8)$

$P_7 = strassen(S_9, S_{10})$

$C_{11} = P_4 + P_5 + P_6 - P_2$

$C_{12} = P_1 + P_2$

$C_{21} = P_3 + P_4$

$C_{22} = P_1 - P_3 + P_5 - P_7$

return C

8) Use #8 to compute $\overset{A}{\begin{bmatrix} 2 & 1 \\ 3 & 2 \end{bmatrix}} \cdot \overset{B}{\begin{bmatrix} 0 & 4 \\ 1 & 3 \end{bmatrix}}$:

$n=2$, so we partition into 4 1×1 s.

here we have $A = \begin{bmatrix} \overset{A_{11}}{2} & \overset{A_{12}}{1} \\ \overset{A_{21}}{3} & \overset{A_{12}}{2} \end{bmatrix}$ $B = \begin{bmatrix} \overset{B_{11}}{0} & \overset{B_{12}}{4} \\ \overset{B_{21}}{1} & \overset{B_{22}}{3} \end{bmatrix}$

$\therefore$

(if $n=1$ return AB)

$S_1 = 4-3=1$, $S_2 = 2+1=3$, $S_3 = 3+2=5$, $S_4 = 1-0=1$, $S_5 = 2+2=4$

$S_6 = 3+0=3$, $S_7 = 1-2=-1$, $S_8 = 1+3=4$, $S_9 = 2-3=-1$ $S_{10} = 0+4=4$

each makes one call back to itself and returns $A \cdot B$.

$P_1 = \text{strassen}(2,1) = 2\cdot1 = 2$, because $n=1$.

$P_2 = \text{strassen}(3,3) = 9$ $P_3 = \text{strassen}(5,0) = 0$

$P_4 = \text{strassen}(2,1) = 2$ $P_5 = \text{strassen}(4,3) = 12$

$P_6 = \text{strassen}(-1,4) = -4$ $P_7 = \text{strassen}(-1,4) = -4$

$C_{11} = \overset{P_4}{2} + \overset{P_5}{12} + \overset{P_6}{-4} - \overset{P_2}{9} = 1$ $C_{12} = \overset{P_1}{2} + \overset{P_2}{9} = 11$

$C_{21} = \overset{P_3}{0} + \overset{P_4}{2} = 2$ $C_{22} = \overset{P_1}{2} - \overset{P_3}{0} + \overset{P_5}{12} - \overset{P_7}{-4} = 18$

$\therefore$ $C = \begin{bmatrix} 1 & 11 \\ 2 & 18 \end{bmatrix}$ is $O(n^2)$

9) $T(n) = \underline{T(n-1)} + n$

$= [T((n-1)-1) + (n-1)] + n$

$= \underline{T(n-2)} + n-1 + n$

$= [T(n-1-2) + n-1-1] + (n-1) + n$

$= T(n-3) + (n-2) + (n-1) + n$

dist. $= T(n-3) + n + n + n - 2 - 1$

$T(n-3) + 3n - 3$

Pattern!

$T(n) = T(n-x) + xn - c$

we get $T(1)$ when $x = (n-1)$

$\therefore$ $T(n) = T(n-(n-1)) + (n-1)n -$

$= T(1) + n^2 + n - c$

$\boxed{T(1) \to \underline{n^2 + n - c}}$

highest order is n^2

thus $O(n^2)$

10) Find a good upper bound: $T(n) = 4T(n/2 + 2) + n$

4^0 $\quad n$

4^1 $\quad T(\frac{n}{2}+c) \quad T(\frac{n}{2}+c) \quad T(\frac{n}{2}+c) \quad T(\frac{n}{2}+c)$

4^2 $\quad T(\frac{n}{4}+c) \quad T(\frac{n}{4}+c) \quad T(\frac{n}{4}+c) \quad T(\frac{n}{4}+c)$

4^3

4^i sub-problems

A sublevel i, we have $T(c + \frac{n}{2^i}) \times 4^i$

cost of one problem at level i is $\boxed{\frac{n}{2^i}}$

Thus, since $T(1)$ is smallest subproblem, we can find i, the sublevel at the bottom, which is the height of the tree by seeing when $\frac{n}{2^i} = 1$.

$$\frac{n}{2^i} = \frac{1}{1} \longrightarrow n = 2^i$$

thus $\log_2$ of both sides gives $\boxed{i = \log_2 n}$

We recall any level has 4^i subproblems.

We can determine total num of subproblems evaluated by 4^i at the bottom sublevel $i = \log_2 n$ where we find $T(1)$.

$$4^i = 4^{\log_2 n} = n^{\log_2 4} = \boxed{n^2} \text{ (because } 2^2 = 4)$$

Thus, a good upper bound is $O(n^2)$.

VERIFY

$T(n) \le 4T(n/2 + c) + n$ $\quad$ ignore constant

$\le 4[4T(n/4) + (n/2)] + n$

$\le 4^2 T(n/4) + 3n$

$\le \cancel{4^2[4T(n/8) + (n}$

$\le 4^3 T(n/8) + 7n \leftarrow \overbrace{n + 2n + 2^2 n \ldots}^{\text{some constant}}$

PATTERN

$$4^r T(n/2^r) + \overbrace{n + 2n + 2^2 n + \ldots 2^{r-1} n}^{\text{constant}}$$

Find where $\frac{n}{2^r} = 1 \quad \therefore \quad n = 2^r, \quad \underline{r = \log_2 n}$

$$\rightarrow \quad 4^r = 4^{\log_2 n} + cn = n^{\log_2 4} + cn$$

$$= n^2 + cn$$

Thus $O(n^2)$!

11) Master Method

1) $T(n) = 2T(n/4) + 1$ $\quad a = 2 \quad b = 4 \quad f(n) = n^0$

~~a > degree of f(n)~~, $2 > 1$ thus

let d be degree of $f(n)$ $\quad T(n) = \Theta(n^{\log_4 2})$

Since $a > b^d$, we have

$(2 > 4^0)$ $\quad \Theta(n^{\log_b a})$ $\quad \boxed{T(n) = \Theta(\sqrt{n})}$

2) $T(n) = 2T(n/4) + \sqrt{n}$ $\quad a = 2 \quad b = 4 \quad f(n) = n^{\frac{1}{2}}$

let d be degree of $f(n) = \frac{1}{2}$

$a = 2$

$b^d = 4^{\frac{1}{2}} = 2$

$\therefore a = b^d$, so $T(n) = \Theta(n^d \log n)$

$\therefore \boxed{T(n) = \Theta(\sqrt{n} \log n)}$

<u>Note: Master Method Laws:</u>

$T(n) = \Theta(n^d)$ if $a < b^d$

$T(n) = \Theta(n^d \log n)$ if $a = b^d$

$T(n) = \Theta(n^{\log_b a})$ if $a > b^d$